```
cat /proc/6141/maps
```

各种安全选择的编译参数如下：

NX：-z execstack / -z noexecstack（关闭 / 开启）

Canary：-fno-stack-protector /-fstack-protector / -fstack-protector-all（关闭 / 开启 / 全开启）

PIE：-no-pie / -pie（关闭 / 开启）

RELRO：-z norelro / -z lazy / -z now（关闭 / 部分开启 / 完全开启）

由 malloc 申请的内存为 chunk

在64位环境下测试，

fastbin

只用了0~5 ，6个 ，大小为0x20,0x30...0x70，

unsorted bin

首个unsorted chunk 指向top chunk ， 每次分配使用bk指针指向的地址，且unlink

- **pre_size :如果物理相邻的前一个地址的chunk 空闲的话，记录前一个chunk的大小，否则存储物理相邻的前一个chunk的数据。前一个chunk为低地址chunk。**
- **size ， 当前chunk的大小， 大小必须是2*SIZE_SZ的整数倍，申请的内存大小不是 2 * SIZE_SZ 的整数倍，会被转换满足大小的最小的 2 * SIZE_SZ 的倍数32位系统中，SIZE_SZ 是 4；64 位系统中，SIZE_SZ 是 8。 该字段的低三个比特位对 chunk 的大小没有影响，它们从高到低分别表示**
    - **NON_MAIN_ARENA，记录当前 chunk 是否不属于主线程，1 表示不属于，0 表示属于。**
    - **IS_MAPPED，记录当前 chunk 是否是由 mmap 分配的。**
    - **PREV_INUSE，记录前一个 chunk 块是否被分配。一般来说，堆中第一个被分配的内存块的 size 字段的 P 位都会被设置为 1，以便于防止访问前面的非法内存。当一个 chunk 的 size 的 P 位为 0 时，我们能通过 prev_size 字段来获取上一个 chunk 的大小以及地址。这也方便进行空闲 chunk 之间的合并。**

- **fd，bk。 chunk 处于分配状态时，从 fd 字段开始是用户的数据。chunk 空闲时，会被添加到对应的空闲管理链表中，其字段的含义如下**
    a. fd 指向下一个（非物理相邻）空闲的 chunk
    b. bk 指向上一个（非物理相邻）空闲的 chunk
    c. 通过 fd 和 bk 可以将空闲的 chunk 块加入到空闲的 chunk 块链表进行统一管理
- **fd_nextsize， bk_nextsize，也是只有 chunk 空闲的时候才使用，不过其用于较大的 chunk（large chunk）。**
    a. fd_nextsize 指向前一个与当前 chunk 大小不同的第一个空闲块，不包含 bin 的头指针。
    b. bk_nextsize 指向后一个与当前 chunk 大小不同的第一个空闲块，不包含 bin 的头指针。
    c. 一般空闲的 large chunk 在 fd 的遍历顺序中，按照由大到小的顺序排列。**这样做可以避免在寻找合适 chunk 时挨个遍历。**

一个已经分配的 chunk 的样子如下。我们称前两个字段称为 chunk header，后面的部分称为 user data。每次 malloc 申请得到的内存指针，其实指向 user data 的起始处。

当一个 chunk 处于使用状态时，它的下一个 chunk 的 prev_size 域无效，所以下一个 chunk 的该部分也可以被当前 chunk 使用。这就是 chunk 中的空间复用。（仅当请求空间向上取整以后对齐8字节才复用）

```
chunk-> +-+-+-+
```

```
+-+-+
        |             Size of chunk, in bytes
|A|M|P|
  mem-> +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-
+-+-+
        |             User data starts here...
.
        .
.
        .             (malloc_usable_size() bytes)
.
next    .
|
chunk-> +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-
+-+-+
        |             (size of chunk, but used for application data)
|
        +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-
+-+-+
        |             Size of next chunk, in bytes
|A|0|1|
        +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-
+-+-+
```

unlink 危险检测机制：

1. double free 机制释放已处理释放状态的chunk
2. 无效的next size : next chunk size 应该在8 bytes 到 arena's total system momory
3. Previous chunk's fd and next chunk's bk should point to currently unlinked chunk. When attacker overwrites fd and bk with free -12 and shellcode address, respectively, free and shellcode address + 8 doesnt point to currently unlinked chunk ('second'). Hence 'glibc malloc' throws up corrupted double linked list error.

当分配的堆块的大小在某一个范围内的话，系统会通过brk方法来分配（brk 会直接拓展原来的堆），但是当分配超级大的堆块时，程序会用mmap方法来分配堆（mmap会单独映射一块内存），mmap分配的堆块和libc之间存在着固定的偏移，因此我们可以推算出来libc的基地址（偏移需要用gdb中的vmmap来计算）

mmap：

PIE 是什么？？

更高级的地址随机化处理

在 ptmalloc，判断当前 chunk 是否是 use 状态的操作如下：

```
#define inuse(p)
 ((((mchunkptr)(((char *) (p)) + chunksize(p)))->mchunk_size) & PREV_INUSE)
```